

Malware Analysis Using REMnux

In this second article of the series, we focus on eliminating XOR patterns. Malware tends to employ XOR patterns for self-encryption/decryption because they obfuscate malicious code which, in turn, helps malware avoid detection.



This article gives you an overview of malware code obfuscation. Nowadays, malware tends to remain hidden during infection and operation, avoiding detection and analysis by security tools. Generally, code obfuscation is malware's best friend. It provides high persistence to the malware samples. Code obfuscation is a technique that uses binary and textual data in an unreadable format that cannot be reverse engineered.

A lot of malware have the capability of self-encryption and high persistence. Most malware use the XOR self-encryption/decryption routine. XOR patterns for obfuscating the malicious code are very similar in working patterns:

- i) Out of a possible 255 keys ranging from 0–255, the attacker picks a 1 byte key
 - ii) The attacker obfuscates the code by encoding it with the 1 byte key value, iteratively
 - iii) The attacker can pick the longer key size to obfuscate the code for better persistence
 - iv) The attacker protects the code (encoded string) by iteratively XORing each byte with the key
- XOR is basically very simple, symmetric and easily

reversible with a single functionality for both encryption and decryption. Encrypted malware is hard to detect by the traditional security solutions like anti-virus software, intrusion detection systems and intrusion prevention systems. The only pattern that can be recognised is the XOR pattern used for the encryption/decryption routine. Here, REMnux is used to analyse the malware samples for possible XOR patterns.

NoMoreXor

NoMoreXor is a command line utility used to analyse and guess the 256 byte XOR key using the frequency analysis technique. The utility is bundled in REMnux and is freely available on Git at <https://github.com/hiddenillusion/NoMoreXor>

The syntax is:

```
NoMoreXor.py -a -o [outputfile.extension] [malwaresample.extension]
```

Here's an example:

```
NoMoreXor.py -a -o malware.hex malware.exe
```